

LangChain & Agentic AI – From Basics to Advanced demo-13 Notes

=====

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(  
    model="gpt-4.1-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
response = llm.invoke(  
    "What is Python? Explain in one sentence."  
)
```

```
#print(type(response))
```

```
#print(response)
```

```
#print(response.content)
```

```
#print("Input Tokens:",  
response.usage_metadata["input_tokens"])
```

```
#print("Output Tokens:",
response.usage_metadata["output_tokens"])

#print("Total Tokens:",
response.usage_metadata["total_tokens"])

print("Model Provider:",
response.response_metadata["model_provider"])

print("Model Name:",
response.response_metadata["model_name"])
```

What is output processing:

taking the model's output and converting or extracting it into the form required by our application.

What is output parser:

It is langchain component that processes model output and converts into a useful output form.

Prompt --> ChatModel ---> ModelOutput ---> Output Parser --> Useful output

Model vs Parser:

Model: Understands the input --> Generates the answer

Parser: Receives the Model output --->processes it -->Returns in required form

StrOutputParser:

It will convert Model output into plain text(like string)

```
from langchain_core.output_parsers import StrOutputParser
```

```
parser = StrOutputParser()
```

```
response = llm.invoke(prompt)
```

```
output = parser.invoke(response)
```

=====

```
import os
```

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(
    model="gpt-4.1-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
parser = StrOutputParser()
```

```
response = llm.invoke(
    "What is Python? Explain in one sentence."
)
```

```
output = parser.invoke(response)
print(type(output))
```

```
=====
```

```
response.content vs StrOutputParser:
```

```
-----
```

response.content ==> Directly access message content in string form

StrOutputParser==> Use a reusable LangChain output-processing component.

Our langchain applications are pipelines mostly, we required to use this parser output to the next component as input in the pipeline.

```
chain = prompt | model | parser | ....
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4.1-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = PromptTemplate.from_template(  
    "Explain {topic} in very simple English with one example."  
)
```

```
parser = StrOutputParser()
```

```
topic = input("Enter topic: ")
```

```
prompt = template.invoke(  
    {  
        "topic": topic  
    }  
)
```

```
response = llm.invoke(prompt)
```

```
result = parser.invoke(response)
```

```
print(result)
```

User Input--->PromptTemplate-->Formatted Prompt --->Chat
Model -->AI Message --->StrOutputParser --->Text/String

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4.1-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = PromptTemplate.from_template(
    """
    Generate 5 {level} interview questions
    about {technology}.

    Use simple English.
    """
)
parser = StrOutputParser()

technology = input("Enter technology: ")
level = input("Enter level: ")

prompt = template.invoke(
    {
        "technology": technology,
        "level": level
    }
)
```

```
response = llm.invoke(prompt)
```

```
result = parser.invoke(response)
```

```
print("\nInterview Questions:")
```

```
print(result)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4.1-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = PromptTemplate.from_template(
```

"""

Give a simple recipe for {dish}.

Requirements:

- List ingredients
- Give step-by-step procedure
- Use very simple English

"""

)

```
parser = StrOutputParser()
```

```
dish = input("Enter dish name: ")
```

```
prompt = template.invoke(
```

```
{
```

```
    "dish": dish
```

```
}
```

```
)
```

```
response = llm.invoke(prompt)
```

```
result = parser.invoke(response)
```

```
print("\nRecipe:")
```

```
print(result)
```

```
=====
```

StrOutputParser does not:

- Create the prompt

- call OpenAI itself

- generate a new AI Answer

- remember conversation history

- create JSON automatically

- validate Pydantic data

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(
    model="gpt-4.1-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} to a {level} learner.
```

```
    Requirements:
```

- Use very simple English
- Give 5 important points
- Give one simple example

```
    """
```

```
)
```

```
parser = StrOutputParser()
```

```
topic = input("Enter topic: ")
level = input("Enter learner level: ")

prompt = template.invoke(
    {
        "topic": topic,
        "level": level
    }
)

response = llm.invoke(prompt)

print("\n-----")
print("TYPE BEFORE PARSING")
print("-----")
print(type(response)) #AI Message

result = parser.invoke(response)
```

```
print("\n-----")
print("TYPE AFTER PARSING")
print("-----")
print(type(result)) #TextAccessor
```

```
print("\n-----")
print("FINAL AI RESPONSE")
print("-----")
print(result)
```

=====

Day 1 Revision:

1. A Chat Model does not normally return only a plain string.

It returns: AIMessage

2. For a simple text response, we can access the generated content using:

response.content

3. An Output Parser works on the output side.

Model



Output Parser

4. StrOutputParser gives us:

String/Text

5. The model generates the answer. The parser processes the answer.

6. response.content can be enough for a small program.

7. StrOutputParser becomes especially useful when building reusable LangChain pipelines.

=====

MCQ 1

What does ChatOpenAI.invoke() normally return?

A. str

B. int

C. AIMessage

D. PromptTemplate

Ans: C

MCQ 2

For our simple text response, which property contains the generated content?

A. response.answer

B. response.content

C. response.output

D. response.value

Ans: B

MCQ 3

What is the main role of an Output Parser?

- A. Train the LLM
- B. Generate the original answer
- C. Process model output
- D. Store API keys

Ans: C

MCQ 4

What does StrOutputParser produce?

- A. Database
- B. Plain string
- C. PDF
- D. New model

Ans: B

MCQ 5

Is StrOutputParser compulsory for every LangChain application?

A. Yes

B. No

Ans: B